



Plataformas de visualización en Python

Ecosistema de visualización en Python

Actualmente, la cantidad de información a la que tenemos acceso proviene de múltiples y diversas fuentes u orígenes: archivos de texto, en formato tabular, imágenes, bases de datos con contenido estructurado o no estructurado.

En ese sentido, pensemos, por ejemplo, en los indicadores de desarrollo mundial que compilan estadísticas relevantes, los cuales permiten hacer comparaciones internacionales en temas de progreso y lucha contra la pobreza. De ellos se han integrado múltiples bases de datos con amplia cobertura cronológica (algunos por más de 50 años para más de 200 economías y 40 grupos de países), divididas en seis grandes temas (The World Bank Group, 2025):

1. Pobreza y desigualdad
2. Personas
3. Ambiente
4. Economía
5. Estados y mercados
6. Enlaces globales

Nota: Estas bases de datos las puedes consultar en el material adicional de este documento.

Estas mediciones son importantes para gobiernos y ONG, ya que, con el monitoreo continuo del desarrollo nacional y regional, se pueden identificar avances o retrocesos que impactan en las políticas públicas. Siendo evidente pensar que la generación de gráficas para representar esta información sería más conveniente en el estudio de tendencias entre tantos datos.

Esta necesidad también se vive en el entorno empresarial, pues la incapacidad de analizar grandes cantidades de información, de manera rápida, puede ocasionar pérdidas económicas y un rezago competitivo, debido a que los problemas no se abordan oportunamente ni se reconocen tendencias emergentes. Con la visualización se podría identificar, por ejemplo, los departamentos funcionales que necesitan mayor atención, los factores que influyen en el comportamiento de los clientes o los volúmenes de ventas e inventarios.

Del mismo gráfico, **tres de las plataformas más utilizadas para crear gráficos no interactivos o estáticos son: matplotlib, la API** (*Application Programming Interface* - Interfaz de Programación de Aplicaciones) **plot de pandas y seaborn**. Veamos en qué consiste cada una de ellas.

Matplotlib

Esta plataforma fue creada por John D. Hunter en 2002 y lanzada en 2003 con el propósito de analizar mejor los datos de electroencefalogramas utilizando gráficos 2D. Fue la primera plataforma de visualización para Python y una de las denominadas bibliotecas núcleo (*core*), sobre la que se construyen varias plataformas de nivel superior. Además, está escrita, principalmente, en Python, hace un uso intensivo de NumPy y permite hacer gráficos 2D o 3D de matrices en Python. Aunque tiene su origen en la emulación de los comandos gráficos de MATLAB, es independiente de esta.

Ventajas

- Una de sus ventajas fundamentales es que es multiplataforma.
- Tiene una API completa y potente que te permite personalizar cualquier atributo de la figura.
- Los contextos de aplicación son muy variados.
- La calidad de publicación de sus gráficos hace que muchas personas la utilicen para generar archivos PostScript para su impresión o inclusión en documentos TeX.
- Los desarrolladores la integran en servidores de aplicaciones web como parte de la interfaz gráfica de usuario o simplemente la emplean de forma interactiva desde un *shell* de Python.

Desventajas

- No es recomendada para crear gráficos dinámicos e interactivos.
- Su sintaxis es muy detallada, por lo que se han desarrollado muchas plataformas que proporcionan una API más simple que cubren las tareas más comunes de manera concisa y conveniente. Algunas de estas se han construido sobre matplotlib, como la API plot de pandas y seaborn.

Pandas

En este punto, ya estás familiarizado con pandas, como una de las herramientas de análisis y manipulación de datos más popular en Python. Además, ya has utilizado sus estructuras (series y *dataframes*) para representar y operar información en formato tabular; sin embargo, pandas también proporciona funciones de trazado a través de su API `.plot()`. Esta interfaz renderiza gráficos estáticos en libretas de Jupyter o para exportar desde Python, con un comando que puede ser tan simple como `df.plot()`.

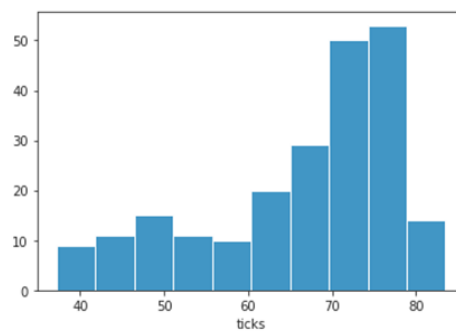
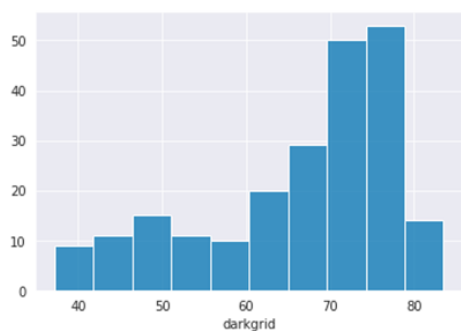
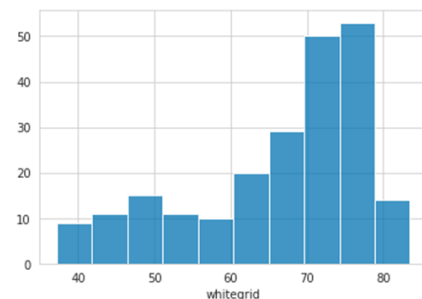
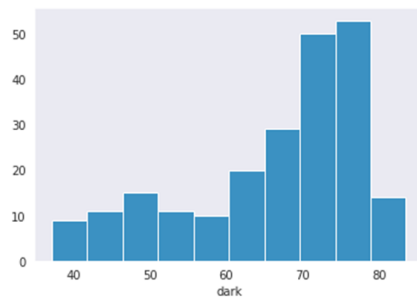
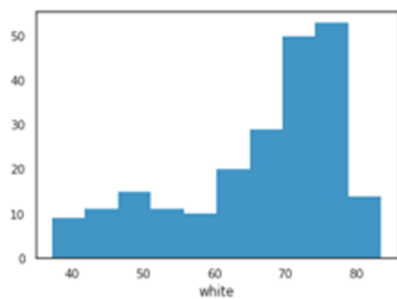
Las funciones de esta API han sido diseñadas para comprender mejor la estructura de los datos, más que para crear gráficos con calidad de publicación; sin embargo, como los gráficos creados en pandas son objetos de matplotlib, es posible emplear las opciones de trazado avanzadas disponibles en esta plataforma núcleo para lograr gráficas presentables e incrementar las posibilidades de personalización.

Seaborn

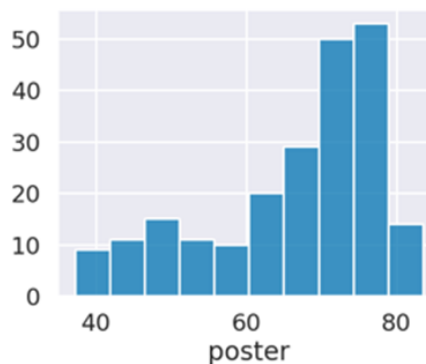
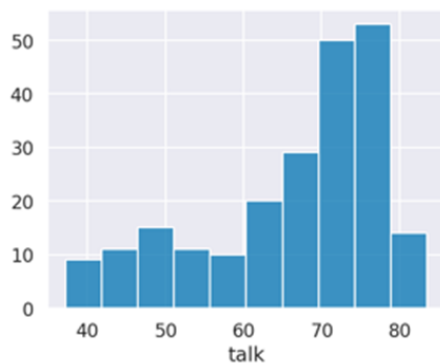
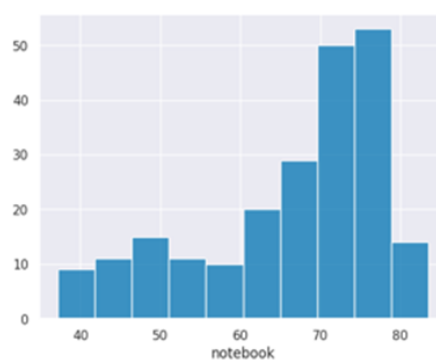
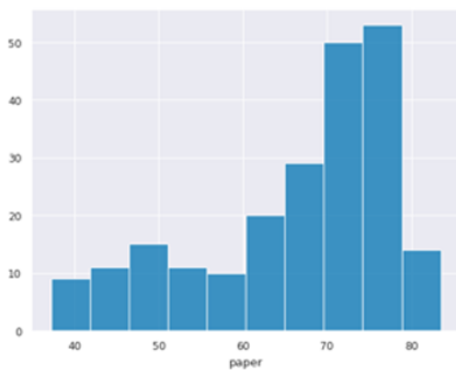
Seaborn es otra de las plataformas de alto nivel que permiten dibujar gráficos estadísticos atractivos e informativos. Además, se integra estrechamente con las estructuras de datos de pandas y posee diversidad de **paletas de colores, estilos y contextos predeterminados**, visualmente atractivos y modernos, facilitando la identificación de patrones y el énfasis en los elementos clave del gráfico.



Paletas en seaborn



Estilos en seaborn



Contextos en seaborn

Esta plataforma también cuenta con una extensa variedad de gráficos para examinar relaciones entre múltiples variables, estadísticas agregadas y de recuento.

Al igual que con pandas, podrás modificar los valores predeterminados de seaborn con las opciones de trazado avanzadas de matplotlib.

Ahora que conoces el panorama de visualización de Python, sabes que tienes muchas opciones para elaborar representaciones gráficas de tus datos, que es justo lo que veremos a continuación.

| Configuraciones, hojas de estilo e interfaces

Matplotlib es una de las plataformas más utilizadas por los científicos en Python, debido a la calidad de los gráficos que produce. Además, es el núcleo o *backend* que soporta las capacidades de pandas y seaborn, que han sido concebidas como **envolturas delgadas** (*wrapper*) por proveer una interfaz más simple.

Enfoquémonos nuevamente en los indicadores de desarrollo mundial (pobreza y desigualdad, personas, ambiente, economía, estados y mercados, enlaces globales) que, debido a la cantidad de información que registran, podría analizarse más fácilmente a través de una gráfica.

Iniciemos con el indicador de dinámica poblacional por país, tomado del archivo **Population.csv** (descárgalo desde el apartado “Archivos adjuntos” de la plataforma) y almacenado en el *dataframe* `population` (265 filas x 66 columnas). Observa que es un *dataframe* no etiquetado en las filas (el índice son las posiciones consecutivas que ocupa cada registro).

	Country Name	Country Code	1960	1961	1962	...	2019	2020	2021	2022	2023
0	Aruba	ABW	54922.0	55578.0	56320.0	...	109203	108587.0	107700	107310.0	107359
1	Africa Eastern and Southern	AFE	130072080.0	133534923.0	137171659.0	...	675950189	694446100.0	713090928	731821393.0	750503764
2	Afghanistan	AFG	9035043.0	9214083.0	9404406.0	...	37856121	39068979.0	40000412	40578842.0	41454761
3	Africa Western and Central	AFW	97630925.0	99706674.0	101854756.0	...	463365429	474569351.0	485920997	497387180.0	509398589
4	Angola	AGO	5231654.0	5301583.0	5354310.0	...	32375632	33451132.0	34532429	35635029.0	36749906
...	...	...	...	...	...	...	...	...	...	...	...
260	Kosovo	XKX	984846.0	1011421.0	1036950.0	...	1788891	1790152.0	1786080	1768096.0	1756366
261	Yemen, Rep.	YEM	5532301.0	5655232.0	5782221.0	...	35111408	36134863.0	37140230	38222876.0	39390799
262	South Africa	ZAF	16440172.0	16908035.0	17418522.0	...	59587885	60562381.0	61502603	62378410.0	63212384
263	Zambia	ZMB	3153729.0	3254086.0	3358099.0	...	18513839	19059395.0	19603607	20152938.0	20723965
264	Zimbabwe	ZWE	3809389.0	3930401.0	4055959.0	...	15271368	15526888.0	15797210	16069056.0	16340822

265 rows x 66 columns

Cabe señalar que el último año reflejado en el archivo es 2023, ya que las dinámicas poblacionales evolucionan con el tiempo, y las actualizaciones importantes únicamente se hacen evidentes después de un período considerable de observación y ajuste.

Para comenzar a graficar, lo primero que debes hacer es importar las plataformas de visualización que emplearás en tus *scripts* de Python.

```
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
```

La función de graficado básica es `plot(x, y)`, que grafica valores de **X** contra **Y** como líneas y/o marcadores. Si consideras un *dataframe* `df` de únicamente dos columnas: una de índice y otra de valores, debes utilizar las siguientes instrucciones para obtener un gráfico de línea equivalente con las tres plataformas:

Con matplotlib	Con pandas	Con seaborn
<code>plt.plot(df.index, df[column])</code>	<code>df.plot()</code>	<code>sns.lineplot(x=df.index, y=df[column])</code>

- En pandas, al utilizar `df.plot()`, el índice del *dataframe* se asigna automáticamente al eje horizontal (**X**), mientras que las columnas de valores se representan en el eje vertical (**Y**). Por lo tanto, no es necesario especificar estos parámetros, ya que pandas realiza esta asignación de forma predeterminada.
- En matplotlib, cuando empleas la función `plt.plot()`, la asignación de los ejes **X** y **Y** es posicional. Esto significa que, por defecto, matplotlib asigna los valores del primer argumento (`df.index`) a **X** y los valores del segundo argumento (`df[column]`) a **Y** sin necesidad de especificarlo explícitamente con nombres de parámetros.
- En seaborn, la forma de asignar los ejes no es posicional, sino que debes utilizar parámetros con nombre: **X=** y **Y=**, es decir, debes especificar explícitamente qué columna del *dataframe* corresponde a cada eje.

Para probar lo anterior, filtremos el *dataframe* `population` para quedarnos únicamente con la información de México en los últimos 10 años:

```
mexico = population.loc[population['Country Name'] ==
'Mexico', '2013':'2023']
```

	2013	2014	2015	2016	2017	...	2019	2020	2021	2022	2023
154	118343573.0	119784261.0	121072306.0	122251351.0	123400057.0	...	125762982.0	126799054.0	127648148.0	128613117.0	129739759.0

Ahora reestructuremos el *dataframe* para que los años queden como índices, utilizando la transpuesta (T), y asignemos nombres a ambas dimensiones.

```
mexico = mexico.T  
mexico.columns = ['Population']  
mexico.index.name = 'Year'
```



Population	
Year	
2013	118343573.0
2014	119784261.0
2015	121072306.0
2016	122251351.0
2017	123400057.0
2018	124573711.0
2019	125762982.0
2020	126799054.0
2021	127648148.0
2022	128613117.0
2023	129739759.0

Grafiquemos el *dataframe* resultante con matplotlib:

```
plt.plot(mexico.index, mexico['Population'])
```

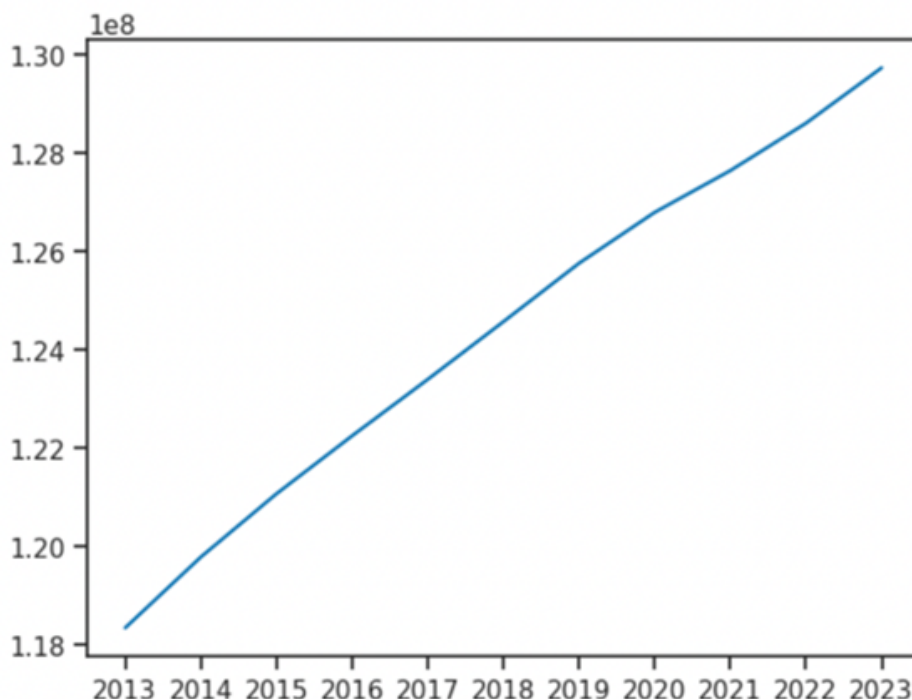
Si estás ejecutando el código en un entorno interactivo (como Colab), te aparecerán dos elementos:

- El valor `<matplotlib.lines.Line2D>` que es una referencia al objeto línea incluido en el gráfico.
- La parte `at 0x7807c5a3bfd0` que es la dirección de memoria donde se encuentra dicho objeto.



```
[<matplotlib.lines.Line2D at 0x7807c5a3bfd0>]
```

```
[<matplotlib.lines.Line2D at 0x7807c5a3bfd0>]
```

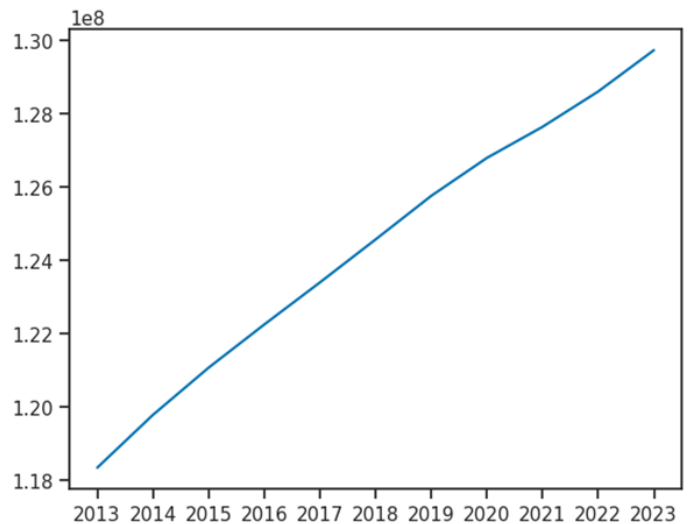


Este tipo de salida aparece porque no le has pedido explícitamente a Python que muestre el gráfico. Por lo que, para evitar este y otros mensajes no deseados, debes llamar explícitamente a `plt.show()`.

Lo anterior es aplicable a todas las plataformas y a todos los tipos de gráficos que generaremos en este módulo. Veamos cómo quedaría:

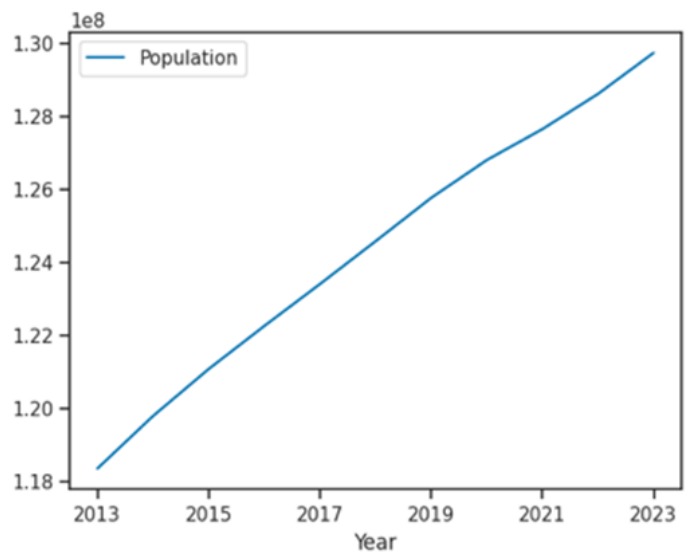
Con matplotlib

```
plt.plot(mexico.index,  
mexico['Population'])  
plt.show()
```



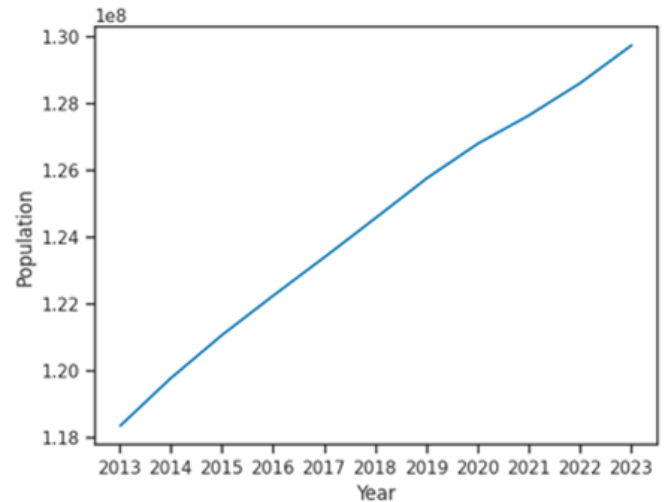
Con pandas

```
mexico.plot()  
plt.show()
```



Con seaborn

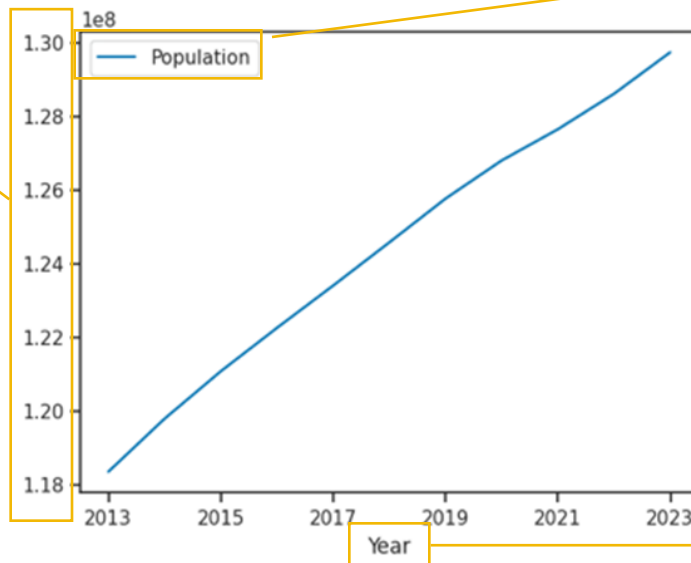
```
sns.lineplot(x=mexico.  
index, y=mexico[ 'Population' ] )  
plt.show()
```



En las tres plataformas se obtienen gráficos muy similares, excepto por la leyenda (únicamente por defecto incluida en pandas) y las etiquetas de los ejes (en *X* para pandas y en ambos ejes para seaborn).

Tomemos el gráfico generado en pandas para identificar sus elementos principales.

yticks: Marcas del eje *Y* con los valores representados en dicho eje.



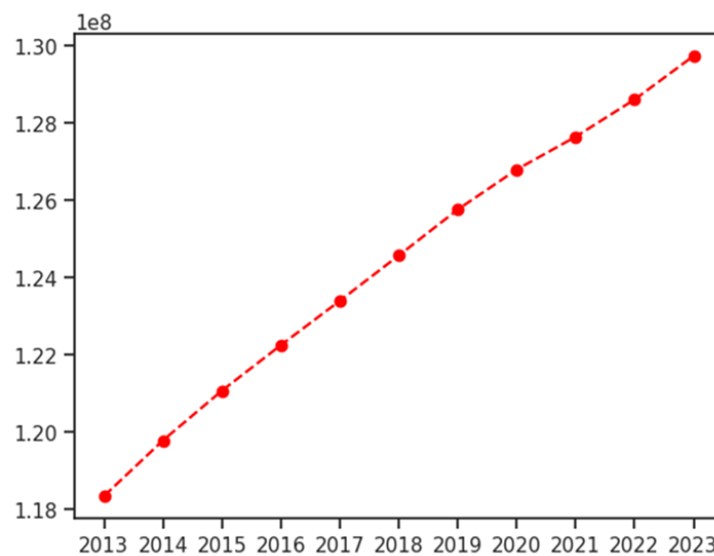
legend: Identifica las distintas series de datos representadas.

xlabel: Etiqueta asociada al eje *X*.

Puedes hacer la personalización de la estética a través de los argumentos provistos por cada plataforma. A continuación, se presentan únicamente algunos, pero puedes consultar la lista completa en el material adicional de este documento.

Matplotlib: color, marker, linestyle,...

```
plt.plot(mexico.index, Mexico['Population'], color='r', linestyle='--', marker='o')
```



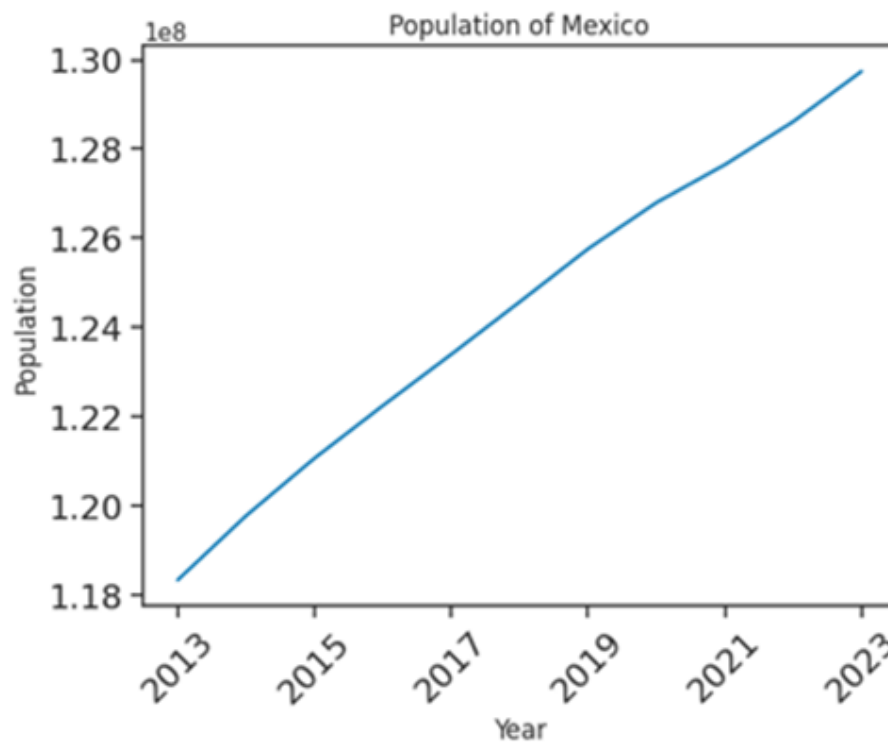
Algunos valores de las propiedades son:

- color: 'b' blue, 'g' green, 'r' red, 'c' cyan, 'm' magenta, 'y' yellow, 'k' black, 'w' white
- linestyle: '-' solid, '--' dashed, '-.' dash-dot, ':' dotted
- marker: '.' point, 'o' circle, 'v' triangle_down, 's' square, '*' star, '+' plus, 'x' x

Nota: Puedes consultar la lista completa de valores en el material adicional de este documento.

Pandas: title, legend, xlabel, ylabel, rot, fontsize,...

```
mexico.plot(title='Population of Mexico',  
            legend=False,xlabel='Year',ylabel='Population',  
            rot=45,fontsize=16)
```



Donde:

- **rot**: Indica la rotación en las marcas del eje **X** en grados.
- **fontsize**: Es el tamaño de la letra en las marcas de ambos ejes.

Nota: Puedes consultar el resto de los parámetros en el material adicional de este documento.

Seaborn: Sus parámetros cobran sentido cuando se realizan agrupaciones semánticas. La relación entre X y Y se puede mostrar para diferentes subconjuntos de datos utilizando los parámetros: `hue`, `size` y `style`.

Donde:

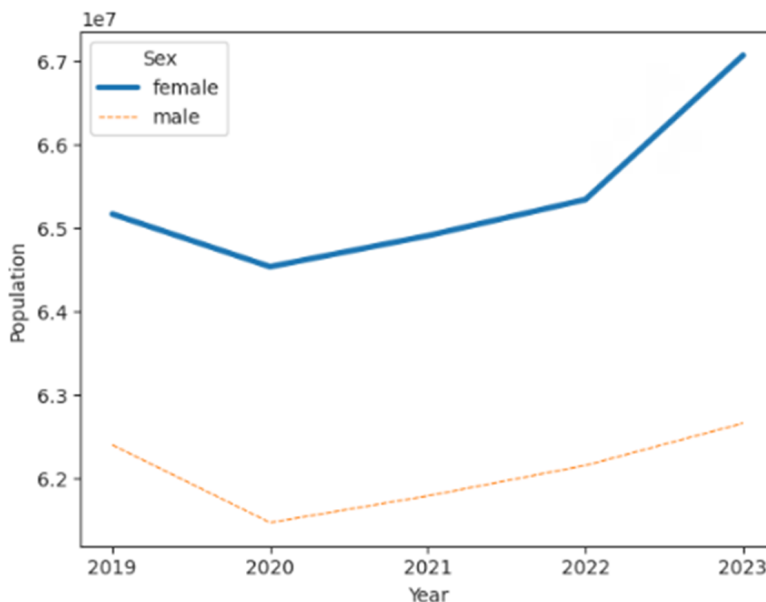
- `hue`: Distingue grupos utilizando colores diferentes.
- `size`: Cambia el tamaño de los puntos según otra variable.
- `style`: Utiliza símbolos o líneas distintas para diferentes categorías.

Por ejemplo, la población podría dividirse por sexo (variable de agrupación semántica), como se muestra en el *dataframe* `gender_mex`:

```
sns.lineplot(x=gender_mex['Year'], y=gender_mex['Population'], hue=gender_mex['Sex'], style=gender_mex['Sex'], size=gender_mex['Sex'])
```



	Year	Sex	Population
0	2019	female	65172136
1	2019	male	62403393
2	2020	female	64540634
3	2020	male	61473390
4	2021	female	64912365
5	2021	male	61792773
6	2022	female	65345824
7	2022	male	62158301
8	2023	female	67075455
9	2023	male	62664304



Aunque en el ejemplo anterior se ha empleado la misma variable (`Sex`), estos parámetros cobran sentido cuando se utilizan variables diferentes.

Nota: Puedes revisar otros parámetros en el material adicional de este documento.

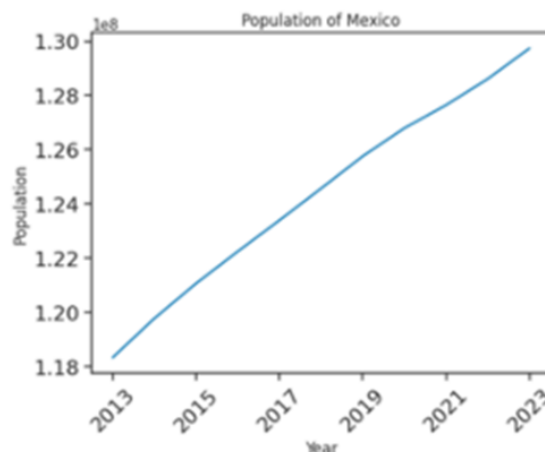
Personalización a través de matplotlib

Como cualquier gráfico creado por pandas y seaborn es un objeto matplotlib y sus métodos `plot()` son únicamente un envoltorio de `plt.plot()`, es posible la personalización de sus propiedades con las siguientes opciones:

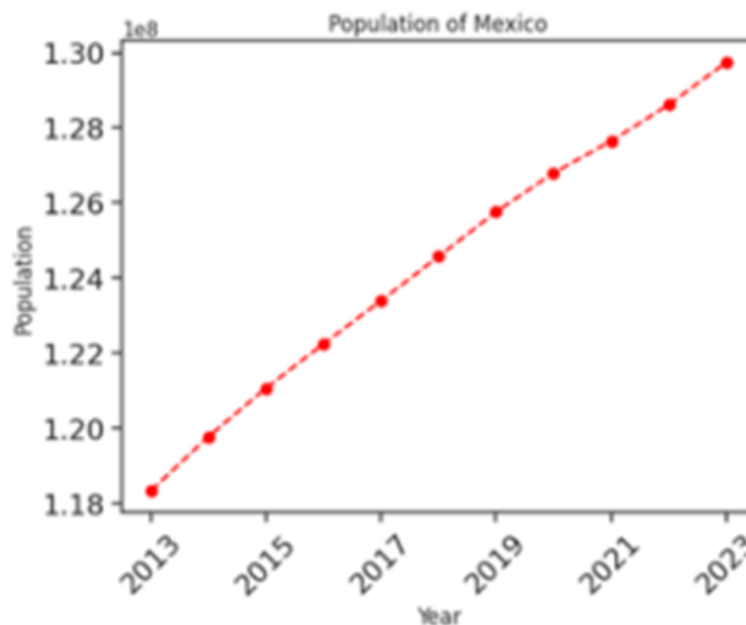
- A través del parámetro `**kwargs` (incluido en la función `plot` de pandas y en `lineplot` de seaborn) que permite pasar argumentos de longitud variable asociados con un nombre.
- Con las funciones de la interfaz pyplot.

Por ejemplo, para personalizar el color, estilo y marcador de la línea, en el gráfico generado con pandas a través de `**kwargs`, basta con incluirlos como se muestra a continuación:

```
mexico.plot(title='Population of Mexico',  
            legend=False,xlabel='Year',ylabel='Population',  
            rot=45,fontsize=16)
```



```
mexico.plot(title='Population of Mexico',  
legend=False,xlabel='Year',ylabel='Population',  
rot=45,fontsize=16,  
color='r',linestyle='--',marker='o')
```



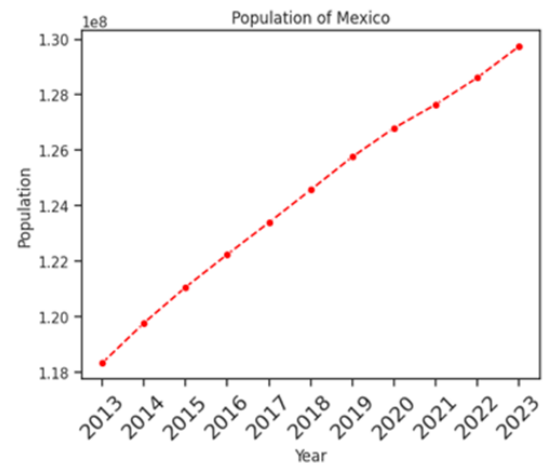
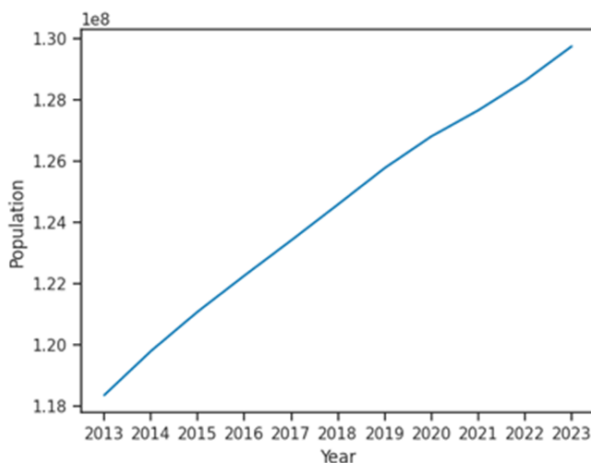
La interfaz pyplot provee muchas funciones para la configuración de los objetos de matplotlib. Las siguientes son algunas de las más utilizadas, pero puedes revisar la lista completa en el material adicional de este documento:

- `Legend()`: Colocar una leyenda en la figura
- `Title()`: Incluir el título
- `Xlabel()` / `ylabel()`: Etiquetar los ejes
- `Xticks()` / `yticks()`: Configurar las marcas de los ejes
- `tick_params()`: Cambiar la apariencia de las marcas de los ejes, sus etiquetas y las líneas de cuadrícula

Por ejemplo, a continuación, se personaliza el gráfico generado con seaborn a través de las funciones de la interfaz pyplot:

```
sns.lineplot(x=mexico.  
index,y=mexico['Population'])
```

```
sns.lineplot(x=mexico.  
index,y=mexico['Population'],  
color='red',linestyle='--  
,marker='o')  
plt.title('Population of  
Mexico')  
plt.xlabel('Year')  
plt.ylabel('Population')  
plt.xticks  
(rotation=45,fontsize=16)
```



Para guardar los gráficos producidos con cualquiera de las plataformas, puedes utilizar la función `savefig` provista por el módulo pyplot de matplotlib:

```
plt.savefig('/content/drive/MyDrive/Colab Notebooks/Proyecto/  
Population_of_Mexico.png')
```

Observa que debes indicar la ruta (si la omites se guardará en el directorio de trabajo actual) y el nombre del archivo con su extensión (png, pdf, ps, eps, jpg...). Podrías incluir además la resolución en puntos por pulgada, la calidad, la orientación, entre otros.

Si necesitas aplicar los mismos parámetros de personalización en todos tus gráficos, configurarlos individualmente puede volverse tedioso. Una forma más eficiente de hacerlo es mediante los ajustes de configuración (*rc - runtime configuration*), que permiten modificar la apariencia y el comportamiento de los gráficos en tiempo de ejecución sin alterar el código base. Estos ajustes se pueden definir utilizando `plt.rcParams`:

```
plt.rcParams['figure.figsize'] = (10, 6) # Tamaño de la figura
plt.rcParams['font.size'] = 18 # Tamaño de la fuente
plt.rcParams['axes.labelsize'] = 18 # Tamaño de etiquetas en ejes
plt.rcParams['axes.titlesize'] = 18 # Tamaño del título
plt.rcParams['xtick.labelsize'] = 14 # Tamaño de etiquetas en eje x
plt.rcParams['ytick.labelsize'] = 14 # Tamaño de etiquetas en eje y
plt.rcParams['legend.fontsize'] = 16 # Tamaño de la leyenda
plt.rcParams['lines.linewidth'] = 4 # Ancho de línea
plt.rcParams['lines.markersize'] = 8 # Tamaño de marcadores
```

Para mostrar todos los parámetros configurables, puedes utilizar:

```
plt.rcParams.keys()
```

A diferencia de matplotlib, pandas y seaborn no tienen un sistema para configurar gráficos globalmente, pero se pueden personalizar mediante argumentos en cada función o empleando las configuraciones de matplotlib.

Hojas de estilos

Los estilos en matplotlib son configuraciones predefinidas que permiten cambiar rápidamente la apariencia de los gráficos sin necesidad de modificar cada parámetro de `plt.rcParams` manualmente. Estas configuraciones se guardan en archivos con extensión `.mplstyle`, denominados hojas de estilo, y se aplican de la siguiente forma:

```
plt.style.use('ggplot') # Aplica el estilo "ggplot"
```

Algunos ejemplos de estilos disponibles son:

- classic
- bmh
- fivethirtyeight
- grayscale

Puedes ver la lista completa con `plt.style.available`.

Si quieres volver a la configuración original de matplotlib, utiliza:

```
plt.style.use('default')
```

En seaborn, puedes emplear `set_theme()` para establecer la configuración global, permitiéndote definir estilo, paleta de colores y contexto, en una sola línea. Es la forma recomendada para personalizar gráficos de manera consistente en toda una sesión. Por ejemplo:

```
sns.set_theme(palette='pastel', style='whitegrid',  
context='talk')
```

Las paletas, estilos y contextos, ya tuviste oportunidad de visualizarlos en la sección anterior. A continuación, te compartimos los valores disponibles para cada parámetro:

- `palette: 'deep', 'deep6', 'muted', 'muted6', 'pastel', 'pastel6', 'bright', 'bright6', 'dark', 'dark6', 'colorblind', 'colorblind6'`
- `style: 'white', 'dark', 'whitegrid', 'darkgrid', 'ticks'`
- `context: 'paper', 'notebook', 'talk', 'poster'`

Interfaces

En matplotlib, existen dos interfaces principales para crear gráficos:

1. Interfaz de estilo MATLAB (`pyplot`)

Es la que hemos utilizado en todos los ejemplos anteriores. Es la forma más común y sencilla de trabajar con matplotlib y se basa en una API de estado global donde los comandos afectan la figura y los ejes actuales sin necesidad de crear explícitamente objetos para ellos. Además, es un enfoque orientado a la simplicidad y rapidez, pero con menos control sobre los objetos que se están manipulando. Por ejemplo, en el siguiente fragmento de código, se establece el título del gráfico sin especificar a qué figura o eje se está refiriendo, ya que `plt` lo gestionará de manera implícita en la figura y en los ejes activos.

```
plt.plot(x, y, marker='o', linestyle='--', color='r')
plt.title('Ejemplo de gráfico con pyplot')
```

2. Interfaz orientada a objetos

Esta ofrece mayor control y flexibilidad, ya que permite trabajar directamente con los objetos (`figure`, `axes`, etc.), sin depender de un estado global.

```
fig, ax = plt.subplots()
ax.plot(x, y, marker='o', linestyle='--', color='r')
ax.set_title('Ejemplo con interfaz orientada a objetos')
```

Por otro lado, con pandas y seaborn puede utilizarse un enfoque similar:

Interfaz de estado	Pandas	<pre>df.plot(x='Año', y='Ventas') plt.title('Ventas por año')</pre>
	Seaborn	<pre>sns.lineplot(df['Año'], df['Ventas']) plt.title('Ventas por año')</pre>
Interfaz orientada a objetos <code>fig, ax1 = plt.subplots()</code>	Pandas	<pre>df.plot(x='Año', y='Ventas', ax=ax1)</pre>
	Seaborn	<pre>sns.lineplot(df['Año'], df['Ventas'], ax=ax1)</pre>

Hasta ahora únicamente hemos visto cómo trazar un gráfico de líneas, para que comprendas las funciones básicas de trazado, la diversidad de sus parámetros y el manejo de la estética; sin embargo, este tipo de gráficos no será adecuado para todos tus contextos ni para todos tus análisis. Afortunadamente, las tres plataformas de estudio disponen de una gran variedad de gráficos. A lo largo de este módulo revisaremos los más importantes:

1. Línea
2. Barras
3. Circular o de sectores (*pie*)
4. Histograma
5. Caja y bigote (*boxplot*)
6. Dispersión (*scatter plot*)
7. Densidad y contorno

En matplotlib y seaborn, cada tipo de gráfico se genera con una función específica, por ejemplo:

- `plt.bar()` para gráficos de barras
- `sns.scatterplot()` para gráficos de dispersión

En cambio, en pandas se utiliza siempre el mismo punto de entrada: `plot()`. Por eso se le conoce como la API `plot()` o la interfaz `plot()` de pandas, ya que permite acceder a distintos tipos de gráficos a partir de un mismo método base.

Este comportamiento es posible gracias al parámetro `kind`, el cual permite especificar el tipo de gráfico que se desea generar. Por ejemplo, `df.plot(kind='bar')` produce un gráfico de barras, mientras que `df.plot(kind='hist')` genera un histograma. Adicionalmente, `df.plot` puede comportarse como un atributo de espacio de nombres para métodos de trazado específicos de la forma `df.plot.<kind>`. Es decir, cada tipo de gráfico tiene un método correspondiente en el descriptor de acceso. Por ejemplo:

- `df.plot(kind='box')` es equivalente a `df.plot.box()`
- `df.plot(kind='bar')` es equivalente a `df.plot.bar()`

Y así para todos los tipos de gráficos. `line` es el tipo por defecto, por eso hasta ahora no ha sido necesario especificarlo en el parámetro `kind`.

Por último, seaborn ofrece varias interfaces a nivel de figura que agrupan los gráficos según el tipo. Por ejemplo, para gráficas categóricas provee la interfaz `catplot`, y a través de ella proporciona acceso a varias funciones especificadas en el parámetro `kind`. Así:

- `sns.catplot(kind='bar')` es equivalente a `sns.barplot()`
- `sns.catplot(kind='count')` es equivalente a `sns.countplot()`

En la siguiente tabla únicamente se muestran las interfaces que implementan los tipos de gráficos que estudiaremos en este módulo (aunque seaborn posee más interfaces y gráficos). Se han omitido los parámetros de las funciones para facilitar la comprensión y se ha marcado en amarillo el tipo de gráfico que se dibujaría por defecto en la interfaz si se omitiera el parámetro `kind`.

Interfaz	Descripción	Llamada a través de la interfaz	Llamada a la función
Gráficos categóricos (catplot)	Proporciona acceso a varias funciones que muestran la relación entre una variable numérica y una o más variables categóricas.	<code>catplot(kind='bar')</code>	<code>barplot()</code>
		<code>catplot(kind='count')</code>	<code>countplot()</code>
		<code>catplot(kind='box')</code>	<code>boxplot()</code>
Gráficos de distribución (displot)	Proporciona acceso a varias funciones que visualizan la distribución univariante o bivariada de los datos.	<code>displot(kind='hist')</code>	<code>histplot()</code>
		<code>displot(kind='kde')</code>	<code>kdeplot()</code>
Gráficos relacionales (relplot)	Proporciona acceso a varias funciones que muestran la relación entre dos variables.	<code>relplot(kind='line')</code>	<code>lineplot()</code>
		<code>relplot(kind='scatter')</code>	<code>scatterplot()</code>

Como habrás podido corroborar a lo largo de esta lección, las configuraciones, estilos e interfaces, ofrecen una amplia variedad de opciones para personalizar y optimizar la visualización de datos. Cada plataforma tiene sus características particulares que permiten adaptarse a diferentes necesidades y preferencias, desde el control detallado y flexible que ofrece matplotlib, hasta la simplicidad y eficacia de seaborn para gráficos estadísticos, y la facilidad de uso e integración que proporciona pandas con su interfaz `plot()`. Comprender cómo utilizar estos elementos es fundamental para crear visualizaciones claras y efectivas que comuniquen de manera precisa los *insights* de los datos.

| Antes de terminar...

Existen muchas plataformas que te permiten trazar datos en el ecosistema de Python. Matplotlib fue la librería inicial de visualización y es considerada una de las más poderosas, pero ello incrementa su complejidad. En cambio, pandas y seaborn son interfaces de trazado superior definidas sobre matplotlib, que buscan simplificar los procesos y configuraciones frecuentes para la generación de gráficos.

Finalmente, te compartimos algunas sugerencias que te ayudarán a elegir la plataforma adecuada de acuerdo con tus necesidades:

- Si te interesa únicamente entender cómo están organizados tus datos, utiliza pandas.
- Si quieres crear gráficos atractivos para comunicar resultados, seaborn es la mejor opción.
- Si necesitas configurar muchos de los parámetros del gráfico, emplea matplotlib.

Recuerda que siempre puedes combinar las plataformas para explotar sus ventajas.

| Material adicional

● Argumentos

Hunter, J., Dale, D., Firing, E., et al. (s.f.). *matplotlib.pyplot.plot*. The Matplotlib development team. Recuperado el 21 de abril de 2025 de: https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.plot.html

● Bases de datos

The World Bank Group. (2025). *Indicators* [Indicadores]. World Bank Group. Recuperado el 21 de abril de 2025 de: <https://data.worldbank.org/indicator>

● Funciones

Hunter, J., Dale, D., Firing, E., et al. (s.f.). *matplotlib.pyplot*. The Matplotlib development team. Recuperado el 21 de abril de 2025 de: https://matplotlib.org/stable/api/pyplot_summary.html

● Matplotlib

The Matplotlib development team. (2025). *Matplotlib: Visualization with Python* [Matplotlib: Visualización con Python]. Matplotlib. Recuperado el 21 de abril de 2025 de: <https://matplotlib.org>

● Pandas

Pandas via. (2025). *Pandas*. Recuperado el 21 de abril de 2025 de: <https://pandas.pydata.org>

● Parámetros

Pandas via. (2024). *Pandas.DataFrame.plot*. Recuperado el 21 de abril de 2025 de: <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.plot.html>

Waskom, M. (2024). *Seaborn.lineplot*. Seaborn. Recuperado el 21 de abril de 2025 de: <https://seaborn.pydata.org/generated/seaborn.lineplot.html>

● Seaborn

Waskom, M. (2021). *Seaborn: statistical data visualization* [Seaborn: visualización de datos estadísticos]. Seaborn. Recuperado el 21 de abril de 2025 de: <https://seaborn.pydata.org>

● Valores

Hunter, J., Dale, D., Firing, E., et al. (2025). *Line styles* [Estilos de línea]. The Matplotlib development team. Recuperado el 21 de abril de 2025 de: https://matplotlib.org/stable/gallery/lines_bars_and_markers/linestyles.html

Hunter, J., Dale, D., Firing, E., et al. (2025). *List of named colors* [Lista de colores nombrados]. The Matplotlib development team. Recuperado el 21 de abril de 2025 de: https://matplotlib.org/stable/gallery/color/named_colors.html

Hunter, J., Dale, D., Firing, E., et al. (2025). *matplotlib.markers*. The Matplotlib development team. Recuperado el 21 de abril de 2025 de: https://matplotlib.org/stable/api/markers_api.html

Nota: Los enlaces a las páginas web incluidos en este apartado fueron consultados y verificados por última vez el 21 de abril de 2025 y su disponibilidad depende del autor.

| Glosario

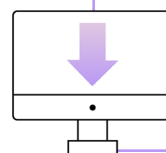
- **Documentos TEX:** Archivo de texto plano que contiene instrucciones de formato y contenido, utilizado para crear documentos científicos y técnicos con alta calidad tipográfica.
- **Estilos:** Configuraciones predefinidas que permiten cambiar rápidamente la apariencia de los gráficos sin necesidad de modificar cada parámetro manualmente.
- **Interfaces:** En el contexto de visualización en Python, el término se refiere a las distintas formas en que el usuario puede interactuar con una biblioteca para construir gráficos. Cada biblioteca ofrece uno o más estilos de interfaz que varían en su nivel de abstracción, flexibilidad y estilo de uso.

| Bibliografía

PyViz. (2019). *Overviews*. Recuperado el 21 de abril de 2025 de: <https://pyviz.org/overviews/index.html>

The World Bank Group. (2025). *World development indicators* [Indicadores de desarrollo mundial]. The World Bank. Recuperado el 21 de abril de 2025 de: <https://datatopics.worldbank.org/world-development-indicators/>

Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 21 de abril de 2025 y su disponibilidad depende del autor.



Descarga este documento en la sección “**Archivos adjuntos**” de la plataforma.